

Course Code: CS2009	Course Name: Design & Analysis of Algorithms
Instructor Name:	
Date of Submission	2nd November 2025

Instructions:

Max Marks: 100

- Make your Assignments on Full Scale Papers, Typed Assignments will not be acceptable:
 - 20% penalty for 1 day late
 - 40% penalty for 2 days late
 - Submission not allowed afterwards
- 1) You are given a set of points in a 2D plane and you have to compute the convex hull of a given set of points using the Jarvis March (Gift Wrapping) algorithm, demonstrating how the orientation function helps identify the most counter clockwise point at each step. **[20 Marks]**

Given Points:

You are given the following six 2D points:

Label	Coordinates (x, y)
A	(0, 3)
B	(2, 2)
C	(1, 1)
D	(2, 1)
E	(3, 0)
F	(0, 0)

Task:

1. Find the starting point — the point with the smallest x-coordinate (if tie, choose smallest y).
2. Use the orientation function to find, at each step, the most counter clockwise point from the current hull point.
3. Repeat until you return to the starting point.
4. List the convex hull points in counter clockwise order.

Orientation Function:

Use the orientation test to determine the turn direction for three points p,q,r:

$$\text{orientation}(p,q,r) = (q_x - p_x)(r_y - p_y) - (q_y - p_y)(r_x - p_x)$$

It tells us the direction of rotation from line segment $p \rightarrow q$ to $p \rightarrow r$:

Interpretation:

If orientation $< 0 \rightarrow$ Clockwise

If orientation $> 0 \rightarrow$ Counterclockwise

If orientation $= 0 \rightarrow$ Collinear

Step	Current Point (p)	Candidate (q)	Checking Point (r)	orientation(p, q, r)	More CCW?	New Candidate
1	F(0,0)	A(0,3)	B(2,2)	?	?	?
2	...	...	...	...	...	...
...						

Students must **fill in this table** for each iteration to determine which point becomes the next hull vertex.

Student Submission Requirements:

Students should:

1. Show orientation computations for each comparison.
2. Fill the dry-run table completely.
3. Draw a diagram labeling all six points and connecting the hull points in CCW order.

Solution:

1) Starting point

Points:

- A (0,3)
- B (2,2)
- C (1,1)
- D (2,1)
- E (3,0)
- F (0,0)

Smallest x is 0 $\rightarrow$ A(0,3) and F(0,0). Tie $\rightarrow$ choose smallest y $\rightarrow$ F(0,0) is the start.

2) Dry-run table (each comparison shown with arithmetic)

Step 1 — $p = F(0,0)$.

Choose initial candidate $q = A(0,3)$. Compare q against other points $r = B, C, D, E$ (alphabetical order).

1. $p=F, q=A, r=B$
 $\text{orientation}(F,A,B) = (0-0)(2-0) - (3-0)(2-0) = 02 - 32 = -6 \rightarrow \text{CW (q stays A)}$
2. $p=F, q=A, r=C$
 $\text{orientation}(F,A,C) = (0-0)(1-0) - (3-0)(1-0) = 01 - 31 = -3 \rightarrow \text{CW (q stays A)}$
3. $p=F, q=A, r=D$
 $\text{orientation}(F,A,D) = (0-0)(1-0) - (3-0)(2-0) = 01 - 32 = -6 \rightarrow \text{CW (q stays A)}$

4. $p=F, q=A, r=E$

$$\text{orientation}(F,A,E) = (0-0)(0-0) - (3-0)(3-0) = 0 - 3*3 = -9 \rightarrow \text{CW (q stays A)}$$

Result Step 1: No r produced a positive orientation, so the most CCW candidate from F is $A(0,3)$.

Hull so far: $F \rightarrow A$

Step 2 — $p = A(0,3)$.

Choose initial candidate $q = B(2,2)$. Compare $r = C,D,E,F$ (alphabetical order).

1. $p=A, q=B, r=C$

$$\text{orientation}(A,B,C) = (2-0)(1-3) - (2-3)(1-0) = 2*(-2) - (-1)*1 = -4 - (-1) = -4 + 1 = -3 \rightarrow \text{CW (q stays B)}$$

(showing substitution another way: $(qx-px) = 2, (ry-py)=1-3=-2; (qy-py)=2-3=-1; (rx-px)=1-0=1$; so $2*(-2) - (-1)*1 = -4 + 1 = -3$)

2. $p=A, q=B, r=D$

$$\text{orientation}(A,B,D) = (2-0)(1-3) - (2-3)(2-0) = 2*(-2) - (-1)*2 = -4 - (-2) = -4 + 2 = -2 \rightarrow \text{CW (q stays B)}$$

3. $p=A, q=B, r=E$

$$\text{orientation}(A,B,E) = (2-0)(0-3) - (2-3)(3-0) = 2*(-3) - (-1)*3 = -6 - (-3) = -6 + 3 = -3 \rightarrow \text{CW (q stays B)}$$

4. $p=A, q=B, r=F$

$$\text{orientation}(A,B,F) = (2-0)(0-3) - (2-3)(0-0) = 2*(-3) - (-1)*0 = -6 - 0 = -6 \rightarrow \text{CW (q stays B)}$$

Result Step 2: No positive orientations $\rightarrow$ next hull vertex is $B(2,2)$.

Hull so far: $F \rightarrow A \rightarrow B$

Step 3 — $p = B(2,2)$.

Choose initial candidate $q = C(1,1)$. Compare $r = A,D,E,F$ (alphabetical order).

1. $p=B, q=C, r=A$

$$\begin{aligned} \text{orientation}(B,C,A) &= (1-2)(3-2) - (1-2)(0-2) \\ &= (-1)1 - (-1)(-2) = -1 - (2) = -3 \rightarrow \text{CW (q stays C)} \end{aligned}$$

(Numerical substitution: $qx-px = 1-2 = -1; ry-py = 3-2 = 1; qy-py = 1-2 = -1; rx-px = 0-2 = -2$; so $(-1)1 - (-1)(-2) = -1 - 2 = -3$.)

2. $p=B, q=C, r=D$

$$\text{orientation}(B,C,D) = (1-2)(1-2) - (1-2)(2-2) = (-1)*(-1) - (-1)*0 = 1 - 0 = 1 \rightarrow \text{CCW} \rightarrow \text{update } q = D(2,1)$$

3. $p=B, q=D, r=E$

$\text{orientation}(B,D,E)$. Now $q=D(2,1)$:

$$\text{orientation}(B,D,E) = (2-2)(0-2) - (1-2)(3-2) = 0*(-2) - (-1)*1 = 0 - (-1) = 1 \rightarrow \text{CCW} \rightarrow \text{update } q = E(3,0)$$

4. $p=B, q=E, r=F$

$$\text{orientation}(B,E,F) = (3-2)(0-2) - (0-2)(0-2) = 1*(-2) - (-2)*(-2) = -2 - 4 = -6 \rightarrow \text{CW (q stays E)}$$

(Note: there is also $r = F$ collinearity check sometimes: $\text{orientation}(B,C,F) = 0$ for the earlier initial $q=C$ case, but because the scanning and updates lead to E being the final candidate, we compare against E and F yields CW and so E remains.)

Result Step 3: Final candidate after scanning is $E(3,0)$.

Hull so far: $F \rightarrow A \rightarrow B \rightarrow E$

Step 4 — p = E (3,0).

Choose initial candidate q = A (0,3). Compare r = B,C,D,F (alphabetical order).

1. p=E, q=A, r=B
 $\text{orientation}(E,A,B) = (0-3)(2-0) - (3-0)(2-3) = (-3)2 - 3(-1) = -6 - (-3) = -6 + 3 = -3 \rightarrow \text{CW (q stays A)}$
2. p=E, q=A, r=C
 $\text{orientation}(E,A,C) = (0-3)(1-0) - (3-0)(1-3) = (-3)1 - 3(-2) = -3 - (-6) = -3 + 6 = 3 \rightarrow \text{CCW} \rightarrow \text{update } q = C(1,1)$
3. p=E, q=C, r=D
 $\text{orientation}(E,C,D) = (1-3)(1-0) - (1-0)(2-3) = (-2)1 - 1(-1) = -2 - (-1) = -2 + 1 = -1 \rightarrow \text{CW (q stays C)}$
4. p=E, q=C, r=F
 $\text{orientation}(E,C,F) = (1-3)(0-0) - (1-0)(0-3) = (-2)0 - 1(-3) = 0 - (-3) = 3 \rightarrow \text{CCW} \rightarrow \text{update } q = F(0,0)$

Result Step 4: Candidate becomes F, which is the starting point $\rightarrow$ algorithm stops.

3) Final convex hull (in counterclockwise order)

Hull vertices found by Jarvis March (CCW):

F (0,0) $\rightarrow$ A (0,3) $\rightarrow$ B (2,2) $\rightarrow$ E (3,0) $\rightarrow$ back to F (0,0)

So the convex hull point set (CCW listing) is:

[F(0,0), A(0,3), B(2,2), E(3,0)]

Points C(1,1) and D(2,1) lie strictly inside that polygon.

- 2) To understand and implement the computation of the KMP failure function (also called the prefix function) using both the naive (brute-force) method and the optimized (linear-time) KMP approach, and to compare their efficiency. **[20 Marks]**

The KMP failure function, $F(j)$, for a pattern $P[0..m-1]$ is defined as: "The size of the largest proper prefix of $P[0..j]$ that is also a proper suffix of $P[0..j]$." A proper prefix of a string is a prefix that is not equal to the entire string itself.

Tasks:

1. Using the above definition, write a brute-force algorithm to compute the failure function $F(j)$ for a given pattern P : $P = \text{"ababaca"}$
 - The algorithm should explicitly test all possible prefix and suffix matches for each position j .
 - Analyze the time complexity of this algorithm.
2. Write the optimized algorithm proposed by Knuth-Morris-Pratt (KMP) to compute the same failure function.
 - Clearly describe how it reuses previously computed values of $F(j)$.
 - Analyze its time complexity.
3. For the pattern P compute the failure function F manually using both methods (brute force and optimized KMP).
 - Show all intermediate steps for each j .
 - Verify that both algorithms produce the same result.

4. Compare the two algorithms in terms of:
 - Time complexity
 - Number of character comparisons
 - Reuse of computed information

Expected Output Format:

j	Substring P[0..j]	Longest Proper Prefix = Suffix	F[j]
0	A	—	0
1	Ab	—	0
2	Aba	a	1
...	...	...	...

Solution:

Pattern

P = "ababaca"

Length m = 7

Indexing (0-based):

j	P[j]
0	a
1	b
2	a
3	b
4	a
5	c
6	a

Part 1 — Brute Force Algorithm

For each position j,
 we consider substring P[0..j] (inclusive),
 and find the largest k such that:
 P[0..k-1] = suffix of P[0..j] of length k
 but k < j+1 (proper prefix).

Brute Force Pseudocode

```
def computeFailure_BruteForce(P):
    m = len(P)
    F = [0]*m
    for j in range(m):
        F[j] = 0
        # check all possible proper prefix lengths
        for k in range(j, 0, -1):
            if P[0:k] == P[j-k+1:j+1]:
                F[j] = k
                break
```

return F

Time Complexity

- For each j , we compare up to $O(j)$ possible prefix lengths, and each comparison can take up to $O(j)$ time.
- So total time $\approx O(m^2)$ in the worst case.

(Brute Force Method)

j	Substring P[0..j]	Prefixes	Suffixes	Longest Proper Prefix = Suffix	F[j]
0	a	—	—	—	0
1	ab	a / b	(none match)	—	0
2	aba	prefixes: a, ab; suffixes: a, ba	match = a	a	1
3	abab	prefixes: a, ab, aba; suffixes: b, ab, bab	match = ab	ab	2
4	ababa	prefixes: a, ab, aba, abab; suffixes: a, ba, aba, baba	match = aba	aba	3
5	ababac	prefixes: a, ab, aba, abab, ababa; suffixes: c, ac, bac, abac, babac	none	—	0
6	ababaca	prefixes: a, ab, aba, abab, ababa, ababac; suffixes: a, ca, aca, baca, abaca, babaca	match = a	a	1

Final failure function:

F = [0, 0, 1, 2, 3, 0, 1]

Part 2 — Optimized KMP Failure Function

Algorithm (Linear Time)

```
def computeFailure_KMP(P):
    m = len(P)
    F = [0]*m
    j = 0 # length of previous longest prefix-suffix
    for i in range(1, m):
        while j > 0 and P[i] != P[j]:
            j = F[j-1] # reuse previously computed prefix length
        if P[i] == P[j]:
            j += 1
        F[i] = j
    return F
```

Time Complexity

- Each character is compared at most twice $\rightarrow O(m)$ time.
- Very efficient compared to the $O(m^2)$ brute-force.

Step-by-Step Dry Run for "ababaca"

i	P[i]	j (before)	Comparison	Action	j (after)	F[i]
0	a	—	—	F[0]=0	—	0

1	b	0	$P[1] \neq P[0] \rightarrow b \neq a$	no prefix match	0	0
2	a	0	$P[2] == P[0] \rightarrow a = a$	j++	1	1
3	b	1	$P[3] == P[1] \rightarrow b = b$	j++	2	2
4	a	2	$P[4] == P[2] \rightarrow a = a$	j++	3	3
5	c	3	$P[5] \neq P[3] \rightarrow c \neq b \rightarrow \text{fallback } j = F[2] = 1 \rightarrow P[5] \neq P[1] (c \neq b) \rightarrow \text{fallback } j = F[0] = 0$	no match	0	0
6	a	0	$P[6] == P[0] \rightarrow a = a$	j++	1	1

Final failure function:

$F = [0, 0, 1, 2, 3, 0, 1]$

Matches brute-force result perfectly.

Part 3 — Verification

j	Substring $P[0..j]$	Longest Proper Prefix = Suffix	$F[j]$
0	a	—	0
1	ab	—	0
2	aba	a	1
3	abab	ab	2
4	ababa	aba	3
5	ababac	—	0
6	ababaca	a	1

Both brute-force and KMP give same result:

$F = [0, 0, 1, 2, 3, 0, 1]$

Part 4 — Comparison

Feature	Brute Force	KMP Optimized
Time Complexity	$O(m^2)$	$O(m)$
Character Comparisons	Up to $m^2/2$ in worst case	$\leq 2m$
Reuse of Computed Info	None	Yes — via $F[j-1]$ fallback
Implementation Simplicity	Easy to code	Slightly complex but efficient
Practical Performance	Poor for long repetitive patterns	Excellent (linear time)

Final Output Table

j	Substring $P[0..j]$	Longest Proper Prefix = Suffix	$F[j]$
0	a	—	0
1	ab	—	0
2	aba	a	1
3	abab	ab	2
4	ababa	aba	3
5	ababac	—	0
6	ababaca	a	1

- 3) Understand, how these problems can be solved using dynamic programming and solve each of the following problem using an example. **[48 Marks]**

- a) The Coin Change Problem is a classic algorithmic problem that involves finding the number of ways to make change for a given amount of money (n) using a set of distinct coin denominations. Each coin denomination can be used unlimited times, and the goal is to determine how many combinations of coins can be used to make up the total amount. **Now solve for S = {1,5,6,8}, Desired Change is 13. [12 Marks]**

Solution:

Manual Step-by-Step Table

Start:

dp = [1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]

Using coin 1

Every amount can be made in exactly 1 way (all 1's).

dp = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]

Using coin 5

For each $j \geq 5 \rightarrow dp[j] += dp[j-5]$

j	dp[j] (before)	dp[j-5]	dp[j] (after)
5	1	dp[0]=1	2
6	1	dp[1]=1	2
7	1	dp[2]=1	2
8	1	dp[3]=1	2
9	1	dp[4]=1	2
10	1	dp[5]=2	3
11	1	dp[6]=2	3
12	1	dp[7]=2	3
13	1	dp[8]=2	3

After coin 5:

dp = [1,1,1,1,1,2,2,2,2,3,3,3,3]

Using coin 6

For each $j \geq 6 \rightarrow dp[j] += dp[j-6]$

j	dp[j] (before)	dp[j-6]	dp[j] (after)
6	2	dp[0]=1	3
7	2	dp[1]=1	3
8	2	dp[2]=1	3
9	2	dp[3]=1	3

10	3	dp[4]=1	4
11	3	dp[5]=2	5
12	3	dp[6]=3	6
13	3	dp[7]=3	6

After coin 6:

dp = [1,1,1,1,1,2,3,3,3,4,5,6,6]

Using coin 8

For each $j \geq 8 \rightarrow dp[j] += dp[j-8]$

j	dp[j] (before)	dp[j-8]	dp[j] (after)
8	3	dp[0]=1	4
9	3	dp[1]=1	4
10	4	dp[2]=1	5
11	5	dp[3]=1	6
12	6	dp[4]=1	7
13	6	dp[5]=2	8

Final dp after all coins:

[1,1,1,1,1,2,3,3,4,4,5,6,7,8]

Step 4 — Final Answer

Number of combinations to make change for 13 using {1, 5, 6, 8} = 8

- b) The Edit Distance, also known as the Levenshtein Distance, measures the similarity between two strings by calculating the minimum number of operations required to transform one series into another. The allowed operations are:
- Insertion: Add a character to the string.
 - Deletion: Remove a character from the line.
 - Substitution: Replace one character with another.

The goal is to find the minimum number of operations needed to equal two strings.

Now solve for str1 = "KITTEN", str2 = "SITTING"

[12 Marks]

Solution:

Initialize Table

str1 = K I T T E N (length 6)

str2 = S I T T I N G (length 7)

We'll make a (7×8) table (including row/col 0).

i\j	""	S	I	T	T	I	N	G
""	0	1	2	3	4	5	6	7
K	1							
I	2							
T	3							
T	4							
E	5							
N	6							

Fill DP Table

Row 1: str1 = "K"

- Compare K vs S → different → $1 + \min(0,1,0)=1$
- So $dp[1][1] = 1$
- Continue for all:

j	S	I	T	T	I	N	G
dp[1][j]	1	2	3	4	5	6	7

Row 1 → [1,2,3,4,5,6,7]

Row 2: str1 = "KI"

- $i=2$, str1[1]=I

Compute cell by cell:

j	str2[j-1]	dp[i][j] reasoning	dp[i][j]
1	S	$\min(1,2,1)+1=2$	2
2	I	match → $dp[1][1]=1$	1
3	T	$\min(3,1,2)+1=2$	2
4	T	$\min(4,2,3)+1=3$	3
5	I	match → $dp[1][4]=4$	4
6	N	$\min(6,4,5)+1=5$	5
7	G	$\min(7,5,6)+1=6$	6

Row 2 → [2,1,2,3,4,5,6]

Row 3: str1 = "KIT"

(str1[2]=T)

j	str2[j-1]	dp[i][j]
1	S	$\min(2,3,2)+1=3$
2	I	$\min(1,3,2)+1=2$
3	T	match → $dp[2][2]=1$
4	T	match → $dp[2][3]=2$
5	I	$\min(4,2,3)+1=3$
6	N	$\min(5,3,4)+1=4$
7	G	$\min(6,4,5)+1=5$

Row 3 → [3,2,1,2,3,4,5]

Row 4: str1 = "KITT" (str1[3]=T)

j	str2[j-1]	dp[i][j]
1	S	$\min(3,4,3)+1=4$
2	I	$\min(2,4,3)+1=3$
3	T	match → $dp[3][2]=2$
4	T	match → $dp[3][3]=1$
5	I	$\min(3,1,2)+1=2$
6	N	$\min(4,2,3)+1=3$
7	G	$\min(5,3,4)+1=4$

Row 4 → [4,3,2,1,2,3,4]

Row 5: str1 = "KITTE" (str1[4]=E)

j	str2[j-1]	dp[i][j]
1	S	min(4,5,4)+1=5
2	I	min(3,5,4)+1=4
3	T	min(2,4,3)+1=3
4	T	min(1,3,2)+1=2
5	I	min(2,2,1)+1=2
6	N	min(3,2,2)+1=3
7	G	min(4,3,3)+1=4

Row 5 → [5,4,3,2,2,3,4]

Row 6: str1 = "KITTEN" (str1[5]=N)

j	str2[j-1]	dp[i][j]
1	S	min(5,6,5)+1=6
2	I	min(4,6,5)+1=5
3	T	min(3,5,4)+1=4
4	T	min(2,4,3)+1=3
5	I	min(2,3,2)+1=3
6	N	match → dp[5][5]=2
7	G	min(4,2,3)+1=3

Row 6 → [6,5,4,3,3,2,3]

Final DP Table

	""	S	I	T	T	I	N	G
""	0	1	2	3	4	5	6	7
K	1	1	2	3	4	5	6	7
I	2	2	1	2	3	4	5	6
T	3	3	2	1	2	3	4	5
T	4	4	3	2	1	2	3	4
E	5	5	4	3	2	2	3	4
N	6	6	5	4	3	3	2	3

- c) The Rod Cutting Problem is a classic optimization problem in computer science and mathematics. It involves finding the best way to cut a long rod into smaller pieces to maximize the total value of the details. Each piece of the rod has a length and an associated weight, and the goal is to determine the combination of cuts that yields the maximum value. **Now solve for length [] = {1, 2, 3, 4, 5, 6, 7, 8}, price [] = {1,5, 8, 9, 10, 16, 18, 20}, Rod Length: 8.** [12 Marks]

Solution:

Given

- Lengths available: 1,2,3,4,5,6,7,8 (implicitly we can cut into any integer length ≤ rod length)

- Prices p for each length (index = length):
 $p[1]=1, p[2]=5, p[3]=8, p[4]=9, p[5]=10, p[6]=16, p[7]=18, p[8]=20$
- Rod length $n = 8$

2) Compute $R[1..8]$ (showing the checks)

We compute $R[j]$ in increasing j . For each j list the candidate values $p[i] + R[j-i]$ and take the maximum.

- $R[0] = 0$
- $j = 1$
 - $i=1: p_1 + R_0 = 1 + 0 = 1 \rightarrow \max = 1, \text{cut}[1]=1$
 $\rightarrow R[1] = 1$
- $j = 2$
 - $i=1: p_1 + R_1 = 1 + 1 = 2$
 - $i=2: p_2 + R_0 = 5 + 0 = 5 \leftarrow \max$
 $\rightarrow R[2] = 5, \text{cut}[2]=2$
- $j = 3$
 - $i=1: 1 + R_2 = 1 + 5 = 6$
 - $i=2: 5 + R_1 = 5 + 1 = 6$
 - $i=3: 8 + R_0 = 8 + 0 = 8 \leftarrow \max$
 $\rightarrow R[3] = 8, \text{cut}[3]=3$
- $j = 4$
 - $i=1: 1 + R_3 = 1 + 8 = 9$
 - $i=2: 5 + R_2 = 5 + 5 = 10 \leftarrow \text{current max}$
 - $i=3: 8 + R_1 = 8 + 1 = 9$
 - $i=4: 9 + R_0 = 9 + 0 = 9$
 $\rightarrow R[4] = 10, \text{cut}[4]=2$
- $j = 5$
 - $i=1: 1 + R_4 = 1 + 10 = 11$
 - $i=2: 5 + R_3 = 5 + 8 = 13 \leftarrow \text{current max}$
 - $i=3: 8 + R_2 = 8 + 5 = 13$
 - $i=4: 9 + R_1 = 9 + 1 = 10$
 - $i=5: 10 + R_0 = 10 + 0 = 10$
 $\rightarrow R[5] = 13, \text{cut}[5]=2$ (we can choose 2 or 3; algorithm picks first max found $\rightarrow 2$)
- $j = 6$
 - $i=1: 1 + R_5 = 1 + 13 = 14$
 - $i=2: 5 + R_4 = 5 + 10 = 15$
 - $i=3: 8 + R_3 = 8 + 8 = 16 \leftarrow \max$
 - $i=4: 9 + R_2 = 9 + 5 = 14$
 - $i=5: 10 + R_1 = 10 + 1 = 11$
 - $i=6: 16 + R_0 = 16 + 0 = 16 \leftarrow \text{tie with } i=3$ (first max chosen may be $i=3$ depending on implementation)
 $\rightarrow R[6] = 16, \text{cut}[6]=3$ (or 6; here we record 3)
- $j = 7$
 - $i=1: 1 + R_6 = 1 + 16 = 17$
 - $i=2: 5 + R_5 = 5 + 13 = 18 \leftarrow \text{current max}$
 - $i=3: 8 + R_4 = 8 + 10 = 18$
 - $i=4: 9 + R_3 = 9 + 8 = 17$
 - $i=5: 10 + R_2 = 10 + 5 = 15$
 - $i=6: 16 + R_1 = 16 + 1 = 17$
 - $i=7: 18 + R_0 = 18 + 0 = 18 \leftarrow \text{tie}$
 $\rightarrow R[7] = 18, \text{cut}[7]=2$ (first max found = 2)
- $j = 8$

- $i=1: 1 + R_7 = 1 + 18 = 19$
 - $i=2: 5 + R_6 = 5 + 16 = 21 \leftarrow \text{current max}$
 - $i=3: 8 + R_5 = 8 + 13 = 21$
 - $i=4: 9 + R_4 = 9 + 10 = 19$
 - $i=5: 10 + R_3 = 10 + 8 = 18$
 - $i=6: 16 + R_2 = 16 + 5 = 21$
 - $i=7: 18 + R_1 = 18 + 1 = 19$
 - $i=8: 20 + R_0 = 20 + 0 = 20$
- $\rightarrow R[8] = 21, \text{cut}[8]=2$ (first i giving 21 is $i=2$)

So the DP array R (max revenues) is:

$R = [0, 1, 5, 8, 10, 13, 16, 18, 21]$
 $j=0 \ 1 \ 2 \ 3 \ 4 \ 5 \ 6 \ 7 \ 8$

3) Reconstruct optimal cuts for length 8

We use $\text{cut}[j]$ repeatedly:

- For $j=8, \text{cut}[8] = 2 \rightarrow$ take piece length 2 (price 5). Remaining length = $8 - 2 = 6$.
- For $j=6, \text{cut}[6] = 3 \rightarrow$ take piece length 3 (price 8). Remaining length = $6 - 3 = 3$.
- For $j=3, \text{cut}[3] = 3 \rightarrow$ take piece length 3 (price 8). Remaining length = $0 \rightarrow$ stop.

So an optimal cutting is: 2, 3, 3

Check revenue: $p[2] + p[3] + p[3] = 5 + 8 + 8 = 21 \rightarrow$ matches $R[8]$.

(Note: there are ties producing the same maximum revenue — e.g., 6+2 or 3+3+2 etc. The DP picks one optimal decomposition; 2,3,3 is valid.)

- d) Given an input string s and a dictionary of words wordDict , determine if s can be segmented into a space-separated sequence of one or more words from wordDict . Words in wordDict can be reused. **Now solve for $\text{wordDict} = \{\text{i, like, ice, cream, icecream, mobile, apple}\}$, Input: ilikeapple .** [12 Marks]

Solution:

We need to determine whether the string

$s = \text{"ilikeapple"}$

can be segmented into a space-separated sequence of valid dictionary words from

$\text{wordDict} = \{\text{i, like, ice, cream, icecream, mobile, apple}\}$

Words in the dictionary can be reused multiple times.

Output can be True/False, and if possible, we'll also show the segmentation.

Step 1 — Initialize

$s = \text{"ilikeapple"}$

$n = 10$ (string length)

Indices:

Index	0	1	2	3	4	5	6	7	8	9	10
Char		i	l	i	k	e	a	p	p	l	e

We use 1-based j to simplify substring $s[i:j]$.

Start with:

$\text{dp} = [\text{True}, \text{False}, \text{False}, \text{False}, \text{False}, \text{False}, \text{False}, \text{False}, \text{False}, \text{False}, \text{False}]$

Step 3 — Fill DP table

$j = 1 \rightarrow s[0:1] = \text{"i"}$

- Check substring "i" ∈ dict? Yes
→ dp[1] = True

dp = [T, T, F, F, F, F, F, F, F, F]

j = 2 → s[0:2] = "il"

Possible splits:

- i=0: "il"
- i=1: dp[1]=True, substring "l"
→ dp[2]=False

j = 3 → s[0:3] = "ili"

- i=0: "ili"
- i=1: dp[1]=True, substring "li"
- i=2: dp[2]=False
→ dp[3]=False

j = 4 → s[0:4] = "ilik"

- i=0: "ilik"
- i=1: dp[1]=True, substring "lik"
- others
→ dp[4]=False

j = 5 → s[0:5] = "ilike"

Now check:

- i=0: "ilike"
- i=1: dp[1]=True, substring "like" ✓ Yes
→ dp[5]=True

So "i like" is valid segmentation.

dp = [T, T, F, F, F, T, F, F, F, F]

j = 6 → s[0:6] = "ilikea"

- i=5: dp[5]=True, substring "a"
→ dp[6]=False

j = 7 → s[0:7] = "ilikeap"

- i=5: dp[5]=True, substring "ap"
→ dp[7]=False

j = 8 → s[0:8] = "ilikeapp"

- i=5: dp[5]=True, substring "app"
→ dp[8]=False

j = 9 → s[0:9] = "ilikeappl"

- i=5: dp[5]=True, substring "appl"
→ dp[9]=False

$j = 10 \rightarrow s[0:10] = \text{"ilikeapple"}$

- $i=5$: $dp[5]=\text{True}$, substring "apple"
 $\rightarrow dp[10]=\text{True}$

"i like apple" works.

Step 4 — Final DP Table

j	Substring ($s[0..j]$)	dp[j]	Reason
0	""	True	base case
1	i	True	"i" in dict
2	il	False	no match
3	ili	False	no match
4	ilik	False	no match
5	ilike	True	"i like"
6	ilikea	False	no match
7	ilikeap	False	no match
8	ilikeapp	False	no match
9	ilikeappl	False	no match
10	ilikeapple	True	"i like apple"

- 4) Take any real-world daily life task and write an algorithm to solve that using dynamic programming approach that is mapped on one of the problems above. **[12 Marks]**

For example: A university student wants to register for courses this semester.

Each course:

- gives a number of credits (academic value or "benefit"), and
- requires a certain number of study hours per week (effort or "cost").

However, the student only has a limited number of study hours they can dedicate weekly (say, due to part-time work, health, or personal limits).

So, the student must select the best combination of courses that:

- maximizes total earned credits (academic value),
- while keeping the total study hours within the weekly limit.

Solution:

A university student wants to register for courses this semester.

Each course:

- gives a certain number of credits (academic value), and
- requires a certain number of study hours per week (effort).

The student can only manage a limited number of study hours (W) each week (e.g., because of a job or personal limit).

The goal is to choose the combination of courses that:

- maximizes total credits, and
- keeps the total study hours $\leq W$.

Course	Study Hours (Cost)	Credits (Value)
C1	2	6
C2	3	10
C3	4	12
C4	5	18

Maximum Weekly Study Hours (W) = 8

This maps directly to the 0/1 Knapsack Problem, where:

- Each course = an item
- Study hours = weight
- Credits = value
- Weekly limit = knapsack capacity (W)

We have 4 courses ($i = 1..4$) and $W = 8$.

Final DP Table (Summary)

$i \backslash w$	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	6	6	6	6	6	6	6
2	0	0	6	10	10	16	16	16	16
3	0	0	6	10	12	16	18	22	22
4	0	0	6	10	12	18	18	22	28

Optimal Selection

$dp[4][8] = 28$ (maximum credits = 28)

To find which courses are selected:

Backtrack:

- $dp[4][8] = 28$ came from including C4 (18) $\rightarrow$ remaining $w = 8 - 5 = 3$
- $dp[3][3] = 10$ came from including C2 (10)

Selected Courses: C2 (3 hrs, 10 credits) and C4 (5 hrs, 18 credits)

Total: 8 hours, 28 credits